

Day 3: Modern JavaScript (ES6+)

In-Depth Reference Guide: Arrow Functions, Asynchronous Timers & Event Loop, Array Methods, Default Parameters, and Spread/Rest Operators

Day 3: Modern JavaScript (ES6+), Timers & Array Methods

Table of Contents

1. Arrow Functions
 - Syntax: Regular vs. Arrow Functions
 - Explicit vs. Implicit Return
 - Returning Functions & Currying
2. Asynchronous JavaScript: Timers & The Event Loop
 - `setTimeout` & Asynchronous Delays
 - The Event Loop & Concurrency Model
 - `setInterval`, `clearInterval` & Countdown Timers
3. The `this` Keyword: Regular vs. Arrow Functions
 - Lexical `this` in Arrow Functions
 - Common Gotchas in Object Methods
4. Modern Array Iteration & Transformation Methods
 - `forEach` vs. `map`
 - `filter`
 - `every` & `some` (Boolean Predicates)
 - `reduce` (The Swiss Army Knife of Arrays)
 - Method Chaining (`filter().map()`)
5. Default Function Parameters

- Syntax & Rules
- Evaluation at Call Time & `undefined` Trigger

6. Spread Operator & Rest Parameters (`...`)

- Spread Operator (`...` for unpacking)
- Rest Parameter (`...` for gathering)
- Shallow Copying & Immutability

7. Day 3 Practice Exercises: Analysis & Complete Solutions

1. Arrow Functions

Introduced in ECMAScript 2015 (ES6), arrow functions provide a concise syntax for writing function expressions and introduce unique scoping behaviors.

Syntax: Regular vs. Arrow Functions

```
// 1. Regular Function Declaration
function multiply(a, b) {
  return a * b;
}

// 2. Regular Function Expression
const multiply = function(a, b) {
  return a * b;
};

// 3. Arrow Function with Explicit Return
const multiply = (a, b) => {
  return a * b;
};

// 4. Arrow Function with Implicit Return
const multiply = (a, b) => a * b;
```

Explicit vs. Implicit Return

Feature	Explicit Return	Implicit Return
Syntax	Curly braces <code>{ ... }</code> + <code>return</code> keyword	No curly braces; expression directly follows <code>⇒</code>
Usage	Required for multiple statements or blocks	Ideal for concise, single-expression calculations
Returning Objects	<code>return { key: value };</code>	<code>({ key: value })</code> (must wrap object in parentheses)

```
// Implicit return of an object literal requires parentheses:  
const createUser = (name, id) ⇒ ({ name, id });  
console.log(createUser("Saif", 101)); // { name: "Saif", id: 101 }
```

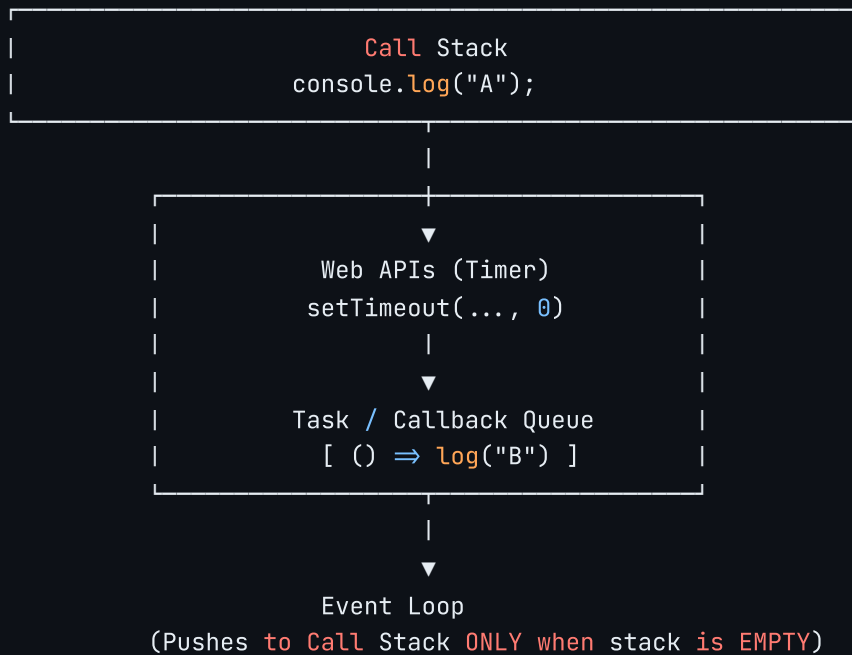
Returning Functions & Currying

Arrow functions can return another function with minimal syntax:

```
// Arrow function returning an inner arrow function  
const square = () ⇒ (num) ⇒ num * num;  
  
const sqr = square();  
console.log(sqr(5)); // 25  
console.log(square()(5)); // 25
```

2. Asynchronous JavaScript: Timers & The Event Loop

JavaScript is **single-threaded**—it has only one Call Stack and executes one operation at a time. Concurrency and asynchronous operations (such as timers, network requests, and DOM events) are coordinated by the browser's runtime environment using the **Event Loop**.



setTimeout & Asynchronous Delays

`setTimeout(callback, delayInMs, ...args)` schedules a callback function to run once after at least `delayInMs` milliseconds.

```
const timerId = setTimeout(() => {
  console.log("Welcome after 2 seconds!");
}, 2000);

// You can cancel a timer before it executes:
// clearTimeout(timerId);
```

The Event Loop & Concurrency Model

Consider the classic interview question:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

Step-by-Step Execution:

1. `console.log("A")` is placed on the Call Stack $\rightarrow$ prints `A` $\rightarrow$ pops off.

2. `setTimeout(..., 0)` is placed on the Call Stack $\rightarrow$ hands the timer off to the browser's **Web APIs** $\rightarrow$ pops off immediately.
3. Web APIs register the 0ms delay and immediately place `()  $\Rightarrow$  console.log("B")` into the **Callback Queue (Task Queue)**.
4. `console.log("C")` enters the Call Stack $\rightarrow$ prints `C` $\rightarrow$ pops off.
5. The Call Stack is now completely **empty**. The **Event Loop** detects this and moves the callback `()  $\Rightarrow$  console.log("B")` from the Queue to the Stack $\rightarrow$ prints `B`.

Final Output:

```
A
C
B
```

Key Rule: Even with `0` milliseconds delay, `setTimeout` cannot execute until the synchronous Call Stack has completely cleared.

setInterval, clearInterval & Countdown Timers

`setInterval(callback, intervalInMs)` schedules a recurring task every `intervalInMs` milliseconds until stopped with `clearInterval(intervalId)`.

Safe Pattern for Automatic Stopping:

```
let count = 1;

const intervalId = setInterval(()  $\Rightarrow$  {
  console.log(count);
  if (count  $\geq$  5) {
    clearInterval(intervalId); // Crucial: prevents infinite running memory leaks
  }
  count++;
}, 1000);
```

Countdown Timer ("10" down to "Liftoff!"):

```
let secondsLeft = 10;

const countdown = setInterval(() => {
  if (secondsLeft > 0) {
    console.log(secondsLeft);
    secondsLeft--;
  } else {
    console.log("Liftoff! 🚀");
    clearInterval(countdown);
  }
}, 1000);
```

3. The `this` Keyword: Regular vs. Arrow Functions

One of the most critical differences between regular functions and arrow functions is how they bind `this`.

Function Type	<code>this</code> Binding Mechanism
Regular Function	Dynamic (Runtime): Bound to the object that calls the function (<code>object.method()</code>).
Arrow Function	Lexical (Static): Does NOT bind its own <code>this</code> ; captures <code>this</code> from the enclosing outer scope at creation time.

Lexical `this` in Arrow Functions

When an arrow function is declared inside an object literal, the enclosing scope is **not** the object—it is the **surrounding execution context** (usually the global window object or module scope).

```
const car = {
  brand: "Mahindra",

  // Regular function: 'this' points to car
  showBrand() {
    console.log(this.brand); // "Mahindra"
  },

  // Arrow function: 'this' points to window / global scope
  showBrandArrow: () => {
    console.log(this.brand); // undefined
  }
};

car.showBrand(); // "Mahindra"
car.showBrandArrow(); // undefined
```

Common Gotchas in Object Methods

Never use arrow functions for object methods that require access to other object properties via

`this`:

```
// BUG:
const player = {
  score: 100,
  getScore: () => {
    console.log(this.score); // undefined (this === window)
  }
};

// FIX: Use standard method shorthand
const fixedPlayer = {
  score: 100,
  getScore() {
    console.log(this.score); // 100
  }
};
```

4. Modern Array Iteration & Transformation Methods

Given array: `const numbers = [4, 8, 15, 16, 23, 42];`

Method	Returns	Mutates Original?	Primary Purpose
<code>forEach()</code>	undefined	No	Executing side effects
<code>map()</code>	New Array	No	Transforming each element
<code>filter()</code>	New Array	No	Selecting matching elements
<code>every()</code>	Boolean	No	Testing if ALL match
<code>some()</code>	Boolean	No	Testing if AT LEAST ONE matches
<code>reduce()</code>	Single Value	No	Accumulating to single result

forEach vs. map

- `forEach`: Iterates over elements strictly for side effects (e.g., logging, updating DOM). It always returns `undefined`.

```
numbers.forEach((num) => {
  console.log(num * 3);
});
```

- `map`: Returns a **new array of identical length**, with each element transformed by the return value of the callback.

```
const squared = numbers.map((num) => num * num);
// [16, 64, 225, 256, 529, 1764]
```

filter

Returns a **new array** containing only elements for which the callback returns a **truthy** value.

```
const greaterThanTen = numbers.filter((num) => num > 10);
// [15, 16, 23, 42]
```

Common Pitfall: In `filter`, return a **boolean condition** (`num > 10`), not the item itself. Any truthy return value includes the element.

every & some (Boolean Predicates)

- `every`: Returns `true` if **every** element satisfies the predicate; stops early (short-circuits) if a false element is found.


```
const allPositive = numbers.every((num) => num > 0); // true
```

- `some`: Returns `true` if **at least one** element satisfies the predicate; stops early when a true element is found.

```
const hasMultipleOfFive = numbers.some((num) => num % 5 === 0); // true (23 is not, b
```

reduce (The Swiss Army Knife of Arrays)

`reduce(callback, initialValue)` iterates through an array and accumulates the values into a single result (number, object, array, etc.).

```
array.reduce((accumulator, currentValue, currentIndex, array) => {  
  return updatedAccumulator;  
}, initialValue);
```

1. Summing Numbers:

```
const totalSum = numbers.reduce((accumulator, current) => {  
  return accumulator + current;  
}, 0);  
console.log(totalSum); // 108
```

2. Finding the Maximum Value (without `Math.max`):

```
const maxVal = numbers.reduce((max, current) => {  
  return current > max ? current : max;  
}, numbers[0]);  
console.log(maxVal); // 42
```

Method Chaining (`filter().map()`)

Because functional array methods return new arrays without mutating the original, you can chain them into expressive, declarative pipelines:

```
// Goal: Square only the even numbers in the array
const evenSquares = numbers
  .filter((num) => num % 2 === 0) // [4, 8, 16, 42]
  .map((num) => num * num);      // [16, 64, 256, 1764]

console.log(evenSquares);
```

5. Default Function Parameters

Default parameters allow formal parameters to be initialized with default values if **no value** or `undefined` is passed.

Syntax & Rules

```
function greetUser(name = "Boss", greeting = "Hello") {
  return `${greeting}, ${name}!`;
}

console.log(greetUser("Saif", "Welcome")); // "Welcome, Saif!"
console.log(greetUser("Saif"));           // "Hello, Saif!"
console.log(greetUser());                 // "Hello, Boss!"
```

Evaluation at Call Time & `undefined` Trigger

- Defaults are evaluated **at call time**, from left to right.
- A default is triggered **only** when an argument is `undefined` (or omitted).
- Passing `null`, `false`, `0`, or `""` will **not** trigger the default:

```
function test(val = 100) { return val; }
test(undefined); // 100 (default used)
test(null);      // null (null is a valid value, default NOT used!)
test(0);         // 0 (default NOT used!)
```

6. Spread Operator & Rest Parameters (...)

The three dots `...` serve two distinct roles depending on context:

- **Spread**: Unpacks elements of an array or object into individual elements.
- **Rest**: Packs multiple comma-separated values into a single array.

Spread Operator (`...` for unpacking)

1. Passing Array Elements into Functions:

```
const arr = [3, 7, 1, 9, 2];
console.log(Math.max(...arr)); // 9 (equivalent to Math.max(3, 7, 1, 9, 2))
console.log(Math.min(...arr)); // 1
```

2. Combining / Merging Arrays:

```
const fruits = ["apple", "banana"];
const veggies = ["carrot", "peas"];

const food = [...fruits, ...veggies];
console.log(food); // ["apple", "banana", "carrot", "peas"]
```

3. Splitting a String into Characters:

```
const letters = [..."javascript"];
// ['j', 'a', 'v', 'a', 's', 'c', 'r', 'i', 'p', 't']
```

Rest Parameter (`...` for gathering)

Rest parameters must be the **last** parameter in a function definition.

```
// Accepts any number of arguments as a real array
const sumAll = (...nums) => {
  return nums.reduce((sum, n) => sum + n, 0);
};

console.log(sumAll(1, 2, 3));           // 6
console.log(sumAll(10, 20, 30, 40));    // 100
console.log(sumAll());                   // 0
```

Positional + Rest Parameters:

```
function introduce(firstName, lastName, ...hobbies) {  
  console.log(`Name: ${firstName} ${lastName}`);  
  console.log(`Hobbies: ${hobbies.join(", ")}`);  
}  
  
introduce("Saif", "Ansari", "Coding", "Reading", "Chess");  
// Output:  
// Name: Saif Ansari  
// Hobbies: Coding, Reading, Chess
```

Shallow Copying & Immutability

Spread creates a **shallow copy** of an object or array:

```
const user = { name: "Amit", age: 25 };  
  
// Shallow copy  
const userCopy = { ...user };  
userCopy.age = 27;  
  
console.log(user.age);    // 25 (original remains unaffected!)  
console.log(userCopy.age); // 27
```

Caution: Nested objects or arrays inside a shallow copy are copied by reference. For deeply nested structures, a deep copy (e.g., `structuredClone(obj)`) is required.

7. Day 3 Practice Exercises: Analysis & Complete Solutions

Below are the refined, bug-free implementations for all exercises in `Day 3/script.js`:

Q1 to Q4: Arrow Functions

```
// Q1. Multiply function variants
function multiplyRegular(a, b) { return a * b; }
const multiplyExplicit = (a, b) => { return a * b; };
const multiplyImplicit = (a, b) => a * b;

// Q2. isEven
const isEven = (num) => num % 2 === 0;

// Q3. greet
const greet = (name) => `Hello, ${name}!`;

// Q4. Curried square function
const square = () => (num) => num * num;
console.log(square()(5)); // 25
```

Q5 to Q8: Timers & Asynchronous Timing

```
// Q5. setTimeout 2s
setTimeout(() => console.log("Welcome!"), 2000);

// Q6. Prediction: "A", "C", "B" (Event Loop ordering)

// Q7. 1 to 5 interval with auto-stop
let num = 1;
const countTimer = setInterval(() => {
  console.log(num);
  if (num ≥ 5) clearInterval(countTimer);
  num++;
}, 1000);

// Q8. Countdown Timer
let count = 10;
const countdown = setInterval(() => {
  if (count > 0) {
    console.log(count);
    count--;
  } else {
    console.log("Liftoff!");
    clearInterval(countdown);
  }
}, 1000);
```

Q9 to Q11: Object Methods and `this`

```
// Q9 & Q10: car object
const car = {
  brand: "Mahindra",
  showBrand() {
    console.log(this.brand); // "Mahindra" (regular function binds to car)
  },
  showBrandArrow: () => {
    console.log(this.brand); // undefined (arrow function captures lexical this)
  }
};

// Q11: Fixing player bug
const player = {
  score: 100,
  getScore() {
    console.log(this.score);
  }
};
player.getScore(); // 100
```

Q12 to Q19: Array Iterators & Transformations

```
const numbers = [4, 8, 15, 16, 23, 42];

// Q12. forEach
numbers.forEach((n) => console.log(n * 3));

// Q13. map
const squared = numbers.map((n) => n * n);

// Q14. filter
const greaterThanTen = numbers.filter((n) => n > 10);

// Q15. every
const allGreaterThanZero = numbers.every((n) => n > 0); // true

// Q16. some
const hasDivisibleByFive = numbers.some((n) => n % 5 === 0); // true

// Q17. reduce (Sum)
const sum = numbers.reduce((acc, n) => acc + n, 0); // 108

// Q18. reduce (Max without Math.max)
const maxVal = numbers.reduce((max, n) => (n > max ? n : max), numbers[0]); // 42

// Q19. Chaining filter & map
const evenSquares = numbers
  .filter((n) => n % 2 === 0)
  .map((n) => n * n); // [16, 64, 256, 1764]
```

Q20 to Q22: Default Parameters

```
// Q20. greetUser
const greetUser = (name = "Friend", greeting = "Hello") => {
  console.log(`${greeting}, ${name}!`);
};

// Q21. calculatePrice
const calculatePrice = (price, tax = 0.05) => price + price * tax;

// Q22. createProfile
const createProfile = (name, age = 18, city = "Unknown") => {
  return `Name: ${name}, Age: ${age}, City: ${city}`;
};

console.log(createProfile("Saif"));
console.log(createProfile("Saif", 20));
console.log(createProfile("Saif", 20, "Jashpur"));
```

Q23 to Q28: Spread & Rest Operators

```
// Q23. Math min / max with spread
const arr = [3, 7, 1, 9, 2];
console.log("Min:", Math.min(...arr)); // 1
console.log("Max:", Math.max(...arr)); // 9

// Q24. Merging arrays
const fruits = ["apple", "banana"];
const veggies = ["carrot", "peas"];
const food = [...fruits, ...veggies];

// Q25. Object copy
const user = { name: "Amit", age: 25 };
const userCopy = { ...user };
userCopy.age = 27;

// Q26. Split string to array
const chars = [..."javascript"];

// Q27. sumAll using rest parameter
const sumAll = (...nums) => nums.reduce((total, n) => total + n, 0);

// Q28. introduce with rest parameter
function introduce(firstName, lastName, ...hobbies) {
  console.log(`Full Name: ${firstName} ${lastName}`);
  console.log(`Hobbies: ${hobbies.join(", ")}`);
}
introduce("Saif", "Ansari", "Programming", "Designing", "Problem Solving");
```

Key Takeaways

1. **Arrow Functions:** Use for compact callbacks and when you want to inherit `this` from the parent context. Avoid for top-level object methods.
2. **Timers & Event Loop:** Synchronous code always runs first; timer callbacks wait in the queue until the call stack is completely idle.
3. **Array Methods:** Choose the right tool:
 - `map` for 1-to-1 transformations.
 - `filter` for subsets.
 - `reduce` for aggregations.
 - `every` / `some` for boolean assertions.

4. **Default Parameters:** Activated strictly when arguments are omitted or passed as `undefined`.

5. **Spread vs. Rest:**

- Spread (`...`) expands/unpacks arrays & objects.
- Rest (`...`) bundles multiple parameters into a single true array.